

CPP102: PROGRAMMING LOGIC AND DESIGN

Week 3 Assessment — Batch 01

Assessment and Application of Program Logic Design

Section A | Monday Class

Department of Computer Engineering

Instructions to Students

Solve all 10 problems below. The problems are divided into three levels of increasing complexity. For every problem, submit all five of the following:

- Input-Process-Output (IPO) Table
- Algorithm (numbered, step-by-step)
- Pseudocode (using START, INPUT, PROCESS, IF, ELSE, DISPLAY, END)
- Flowchart (correct standard symbols)
- Short explanation (1-2 sentences on the logic used)

Python code is not required at this stage. You are assessed only on your algorithm, pseudocode, and flowchart logic. Python implementation will be introduced in a later lesson.

Flowcharts may be created using Mermaid, diagrams.net, MS Word shapes, PowerPoint shapes, or hand-drawn and scanned/photographed, as long as correct standard flowchart symbols are used.

This is Batch 01. The problems below use their own context, numbers, and variable names. Solve only the problems assigned to your batch — do not copy the wording or answers of another batch.

Repository and File Organization

Create a repository (or project folder) named:

`cpp102-week3-batch01-lastname`

Replace "lastname" with your actual last name. Inside it, save the following files:

- README.md
- problem01_algorithm.txt
- problem01_pseudocode.txt
- problem01_flowchart.md (or image)
- problem02_algorithm.txt
- problem02_pseudocode.txt
- problem02_flowchart.md (or image)
- problem03_algorithm.txt
- problem03_pseudocode.txt
- problem03_flowchart.md (or image)
- problem04_algorithm.txt
- problem04_pseudocode.txt
- problem04_flowchart.md (or image)
- problem05_algorithm.txt
- problem05_pseudocode.txt
- problem05_flowchart.md (or image)
- problem06_algorithm.txt
- problem06_pseudocode.txt

- problem06_flowchart.md (or image)
- problem07_algorithm.txt
- problem07_pseudocode.txt
- problem07_flowchart.md (or image)
- problem08_algorithm.txt
- problem08_pseudocode.txt
- problem08_flowchart.md (or image)
- problem09_algorithm.txt
- problem09_pseudocode.txt
- problem09_flowchart.md (or image)
- problem10_algorithm.txt
- problem10_pseudocode.txt
- problem10_flowchart.md (or image)
- screenshots/ (folder containing Git evidence screenshots)

README.md must include:

- Student name
- Section
- Batch number: Batch 01
- Class schedule: Monday
- Course: CPP102 — Programming Logic and Design
- Week 3 assessment title: Assessment and Application of Program Logic Design
- List of completed problems (Problem 1 to Problem 10)
- Reflection of at least 3 sentences
- GitHub repository link

Git Workflow

First-Time Setup

1. Initialize the repository.

```
git init
```

2. Set your Git commit identity for this repository.

```
git config user.name "Student Full Name"  
git config user.email "student@email.com"
```

3. Verify that the identity was set correctly.

```
git config user.name  
git config user.email
```

4. Check the current state of your files.

```
git status
```

5. Stage all your files.

```
git add .
```

6. Commit with a clear message.

```
git commit -m "Add Week 3 Batch 01 program logic assessment"
```

7. Set the branch name and connect to GitHub.

```
git branch -M main  
git remote add origin https://github.com/yourusername/cpp102-week3-  
batch01-lastname.git
```

8. Push to GitHub.

```
git push -u origin main
```

Regular Update Workflow

1. Always pull first to get the latest version of your repository.

```
git pull
```

2. Check the current state of your files.

```
git status
```

3. Stage your changes.

```
git add .
```

4. Commit with a clear message.

```
git commit -m "Update Week 3 Batch 01 assessment solutions"
```

5. Push your changes to GitHub.

```
git push
```

Take a screenshot after running git status and after a successful git commit. Save both in your screenshots/ folder.

Reminders

- Always pull before starting an update.
- Always verify your Git name and email before committing on a shared computer.

- Always commit before pushing.
- Avoid editing directly on GitHub unless instructed.

Assessment Problems

LEVEL 1: BASIC LINEAR PROBLEMS

Simple input, process, and output — no decisions.

Problem 1: Snack Money Total

Andrei received an allowance of num1 pesos from his mother and num2 pesos from his father. Create a logic design that asks for both amounts, adds them, and displays the total snack money.

Problem 2: Garden Plot Area

A rectangular garden plot has a given length and width. Create a logic design that asks for the length and width, computes the area, and displays the result.

Problem 3: Weekly Wage

A part-time working student is paid an hourly rate. Create a logic design that asks for the hourly rate and the number of hours worked, computes the gross pay, and displays the result.

Problem 4: Quiz Average

A student has three quiz scores. Create a logic design that asks for the three scores, computes the average, and displays the result.

LEVEL 2: MULTIPLE-INPUT COMPUTATION PROBLEMS

Several inputs combined through more than one arithmetic operation — still no decisions.

Problem 5: Canteen Order Total

A student ordered three different items at the canteen. Each item has its own price and quantity. Create a logic design that asks for the price and quantity of each of the three items, computes the total bill, and displays the result.

Problem 6: Overtime Pay Computation

An employee is paid an hourly rate for regular hours and 1.5 times that rate for overtime hours. Create a logic design that asks for the hourly rate, the regular hours worked, and the overtime hours worked, computes the total gross pay, and displays the result.

Problem 7: Final Grade Computation

A student's final grade is based on a quiz average, an exam score, and a project score, weighted at 30%, 50%, and 20% respectively. Create a logic design that asks for the three scores, computes the final grade, and displays the result.

LEVEL 3: DECISION PROBLEMS

IF/ELSE logic producing two or three possible outputs.

Problem 8: Passed or Failed

A student's final grade determines whether the student passed the course. Create a logic design that asks for the final grade and displays "Passed" if the grade is 75 or above, or "Failed" otherwise.

Problem 9: Temperature Reading Classification

A weather station records a temperature reading in degrees Celsius. Create a logic design that asks for the temperature and displays "Above Freezing" if it is greater than 0, "Below Freezing" if it is less than 0, or "Exactly Freezing" if it is exactly 0.

Problem 10: Locker Number Check

Every student is assigned a locker number. Create a logic design that asks for a locker number and displays "Even" if the number is divisible by 2, or "Odd" otherwise.

Grading Rubric (100 Points)

Criterion	Full Marks	Partial Marks	No Credit
Input-Process-Output Analysis (15 pts)	Full (15 pts): All 10 problems have complete, correct IPO tables — inputs, process, and output are all clearly and correctly identified.	Partial (7-8 pts): Most IPO tables are present and mostly correct, but a few problems have incomplete or minor errors.	No Credit (0 pts): IPO tables are missing, or input/process/output are consistently incorrect or absent.
Algorithm Correctness and Completeness (20 pts)	Full (20 pts): All 10 algorithms are correctly numbered, logically complete from Start to End, and correctly solve each problem, including proper decision steps where required.	Partial (10 pts): Most algorithms are logically correct but have minor gaps, missing steps, or small logic errors in one or two problems.	No Credit (0 pts): Algorithms are missing, incomplete, or do not solve the given problems.
Pseudocode Correctness and Readability (20 pts)	Full (20 pts): All 10 pseudocode solutions correctly use the required keywords (START, INPUT, PROCESS, IF, ELSE, DISPLAY, END), are properly indented, and match the corresponding algorithm.	Partial (10 pts): Pseudocode is present for most problems but has inconsistent keyword usage, indentation issues, or does not fully match the algorithm.	No Credit (0 pts): Pseudocode is missing or does not follow the required keywords and structure.
Flowchart Logic and Correct Symbol Usage (25 pts)	Full (25 pts): All 10 flowcharts use correct standard symbols (terminal, input/output, process, decision), show correct logic flow, and match the algorithm and pseudocode.	Partial (12 pts): Most flowcharts are present and mostly correct, but a few use incorrect symbols or contain minor logic errors.	No Credit (0 pts): Flowcharts are missing, use incorrect symbols throughout, or do not represent the correct logic.
Git/GitHub Evidence and File Organization (10 pts)	Full (10 pts): Repository is correctly named, all required files are present and properly organized, and screenshots clearly show git status and a successful commit.	Partial (5 pts): Repository and files are present but incompletely organized, or a required screenshot is missing.	No Credit (0 pts): No repository, missing files, or no evidence of Git usage.
Completeness, Neatness, and	Full (10 pts): All 10 problems are	Partial (5 pts): Some problems or files are	No Credit (0 pts): Submission is largely

Criterion	Full Marks	Partial Marks	No Credit
README Reflection (10 pts)	complete, files are neatly organized and labeled, and README.md contains all required information plus a reflection of at least 3 sentences.	missing or disorganized, or the README is incomplete, or the reflection is shorter than required.	incomplete or disorganized, or the README is missing.

Total: 100 points.